Learning From Human Feedback
LLM Alignment

Nikolay Karpachev
13.05.2026

RL outside games

NLP Training / Inference Regimes

- Supervised seq2seq learning:

$$P(y_{t+1}|x, y_{0:t}), \qquad y_{0:t} \sim reference$$

- Inference

$$P(y_{t+1}|x, \hat{y}_{0:t}), \qquad \hat{y}_{0:t} \sim \textbf{???}$$

NLP Training / Inference Regimes

- Supervised seq2seq learning:

$$P(y_{t+1}|x, y_{0:t}), \qquad y_{0:t} \sim reference$$

- Inference

$$P(y_{t+1}|x, \hat{y}_{0:t}), \qquad \hat{y}_{0:t} \sim \textbf{???}$$

If model ever makes something that isn't in data,
It gets volatile from next time-step!

NLP Training / Inference Regimes

- Supervised seq2seq learning:

$$P(y_{t+1}|x, y_{0:t}), \qquad y_{0:t} \sim reference$$

- Inference

$$P(y_{t+1}|x, \hat{y}_{0:t}), \qquad \hat{y}_{0:t} \sim \text{???}$$

“Exposure bias”

If model ever makes something that isn't in data,
It gets volatile from next time-step!

Issue 1: Exposure bias

- Need good training data
  - Abundant
  - Few biases and noise
- Need a perfect network to extrapolate training set

Issue 1: Exposure bias

- Need good training data
  - Abundant
  - Few biases and noise
- Need a perfect network to extrapolate training set

Spoiler: Most of the time we don’t. Too bad.

Issue 2: Crossentropy loss

Reality: sometimes there is more than one correct translation (image caption, summary, whatever)

Source: 在 找 给 家里 人 的 礼物.

Versions:
i 'm searching for some gifts for my family.
i want to find something for my family as presents.
i 'm about to buy some presents for my family.
i 'd like to buy my family something as a gift.
i 'm looking for a present for my family.
...

Issue 2: Crossentropy loss

Reality: sometimes there is more than one correct translation (image caption, summary, whatever)

Source: 在 找 给 家里 人 的 礼物.

Versions:	Model 1 **p(y|x)**	Model 2 **p(y|x)**
(version 1)	1e-2	0.99
(version 2)	2e-2	1e-100
(version 3)	1e-2	1e-100
(all rubbish)	0.96	0.01

Question: which model has better Mean log p(y|x) ?

not in data

This one. While it predicts 96% rubbish

Issue 3: Training data

Two kinds of datasets:

Big enough, but suboptimal R(x,y)

- **Large raw data**
  - twitter, open subtitles, books, bulk logs
  - 10^6-8 samples, http://opus.nlpl.eu/OpenSubtitles.php

- **Small clean data**
  - moderated logs, assessor-written conversations
  - 10^2~4 samples

Near-optimal R(x,y), but too small

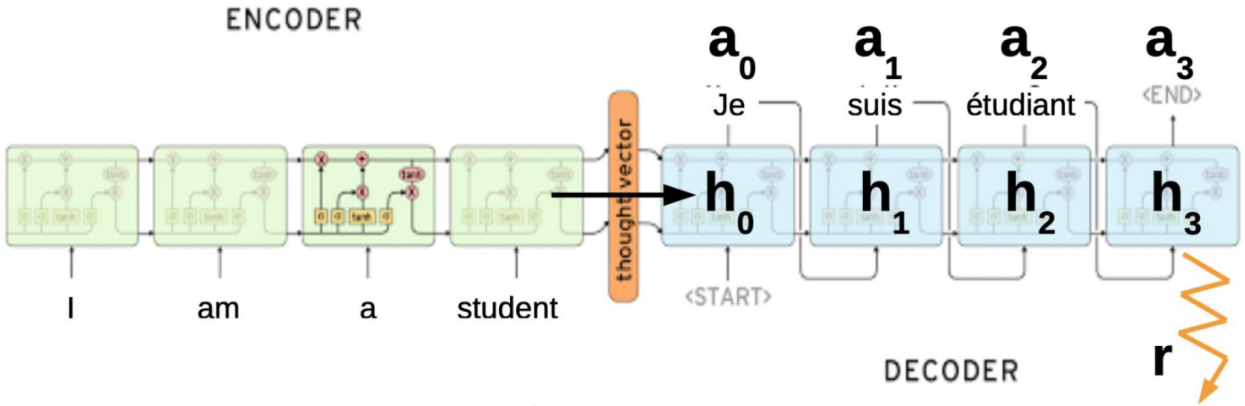
Can we do better?

Key idea: frame structured prediction problem as RL task

Pros:

- Can optimize any objective function directly (not just probability of correct stuff)
  - User feedback (discrete scores)
  - External reward model
  - Practically, any black-box stuff
- Inference data is the new training data!

Seq2Seq as POMDP



Hidden state **s** = translation/conversation state
Initial state **s** = encoder output
Observation **o** = previous words
Action **a** = write next word
Reward **r** = domain-specific reward (e.g. BLEU)

Training with Policy Gradient

Our objective:

Reward
(e.g. BLEU)

$$J = \mathop{E}_{\substack{s\sim p(s)\\ a\sim\pi_\theta(s|a)}} R(s,a) = \int_s p(s) \int_a \pi_\theta(a|s) R(s,a)\, da\, ds$$

parameters are
hidden here

We can approximate the expectation with mean:

$$J \approx \frac{1}{N} \sum_{i=0}^{N} R(s,a)$$

Training with Policy Gradient

Our objective:

$$J = \underset{\substack{s\sim p(s)\\ a\sim\pi_\theta(s|a)}}{E} R(s,a) = \int_s p(s) \int_a \pi_\theta(a|s)\, R(s,a)\, da\, ds$$

$$\nabla J = \int_s p(s) \int_a \nabla \pi_\theta(a|s)\, R(s,a)\, da\, ds$$

Expectation is lost!

We don't know how to compute the gradient w.r.t. parameters

Training with Policy Gradient

Problem: we need gradients on parameters

$$J = \mathop{E}_{\substack{s\sim p(s)\\ a\sim\pi_\theta(s|a)}} R(s,a) = \int_s p(s) \int_a \pi_\theta(a|s)\, R(s,a)\, da\, ds$$

Potential solution: Finite differences

$$\nabla J \approx \frac{J_{\theta+\epsilon} - J_\theta}{\epsilon}$$

Very noisy, especially if both J are sampled

Training with Policy Gradient

Problem: we need gradients on parameters

$$J = \underset{\substack{s\sim p(s)\\ a\sim\pi_\theta(s|a)}}{E} R(s,a) = \int_s p(s) \int_a \pi_\theta(a|s)\, R(s,a)\, da\, ds$$

Wish list:

- Analytical gradient
- Easy/stable approximations

Log-derivative trick

Simple math question:

$$\nabla \log \pi(z) = ???$$

Log-derivative trick

Simple math question:

$$\nabla \log \pi(z) = ???$$

$$\pi \cdot \nabla \log \pi(z) = \nabla \pi(z)$$

Policy Gradient Approximation

Algorithm: Monte-Carlo Policy Gradient (REINFORCE)

$$\mathcal{J}(\theta) = \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi(a|s;\theta) Q_\pi(s,a)$$

$$\nabla \mathcal{J}(\theta) = \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \nabla \pi(a|s;\theta) Q_\pi(s,a)$$

$$= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi(a|s;\theta) \frac{\nabla \pi(a|s;\theta)}{\pi(a|s;\theta)} Q_\pi(s,a)$$

by log-derivative trick

$$= \sum_{s \in \mathcal{S}} d(s) \sum_{a \in \mathcal{A}} \pi(a|s;\theta) \nabla \ln \pi(a|s;\theta) Q_\pi(s,a)$$

$$= \mathbb{E}_{\pi_\theta}[\nabla \ln \pi(a|s;\theta) Q_\pi(s,a)]$$

this expectation can be estimated by samples of episodes

Policy Gradient

$$\nabla J = \int_s p(s) \int_a \nabla \pi_\theta(a|s) R(s,a) da\, ds$$

$$\nabla J \approx \frac{1}{N} \sum_{i=0}^{N} \nabla \log \pi_\theta(a|s) \cdot R(s,a)$$

Supervised Learning vs. Policy Gradient

Supervised learning:

$$\nabla llh = \underset{s, a_{opt} \sim D}{E} \nabla \log \pi_\theta (a_{opt} | s)$$

Policy gradient:

$$\nabla J = \underset{\substack{s \sim d(s) \\ a \sim \pi(a|obs(s))}}{E} \nabla \log \pi_\theta (a|s) Q(s,a)$$

Question: what is different? (apart from Q(s, a))

Supervised Learning vs. Policy Gradient

Supervised learning:

$$\nabla llh = \underset{s, a_{opt} \sim D}{E} \nabla \log \pi_\theta (a_{opt} | s)$$

reference

Policy gradient:

$$\nabla J = \underset{\substack{s \sim d(s) \\ a \sim \pi(a|obs(s))}}{E} \nabla \log \pi_\theta (a|s) Q(s,a)$$

generated

Supervised Learning vs. Policy Gradient

Supervised learning:

- Need (near-)optimal dataset
- Trains on reference sessions

Policy gradient:

- Need ~some data and reward function
- Trains on its own output

Supervised Learning vs. Policy Gradient

Supervised Learning	**Reinforcement Learning**
Need good reference (y_opt)	Need reward function
If model is *imperfect* [and **it is**], training: P(y_next|x,y_prev_ideal) prediction: P(y_next|x,y_prev_predicted)	Model learns to improve current policy. If policy is pure random, local improvements are unlikely to produce good translation.

Supervised Learning vs. Policy Gradient

Supervised Learning	**Reinforcement learning**
+ Rather simple	+ **Cold start problem**
+ Small variance	+ Large variance (so far)
– Need good reference (y_opt)	– Only needs x and r(s,a)
– **Distribution shift:** different **h** distribution when training vs generating	– No **distribution shift**

Supervised Learning vs. Policy Gradient

Supervised Learning

pretraining

+ Rather simple
+ Small variance

– Need good reference (y_opt)
– **Distribution shift:**
different **h** distribution
when training vs generating

Reinforcement learning

finetuning

+ **Cold start problem**
+ Large variance (so far)

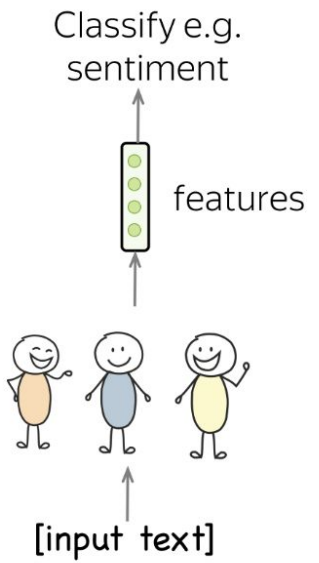
– Only needs x and r(s,a)
– No **distribution shift**

Part 2: LLM Alignment

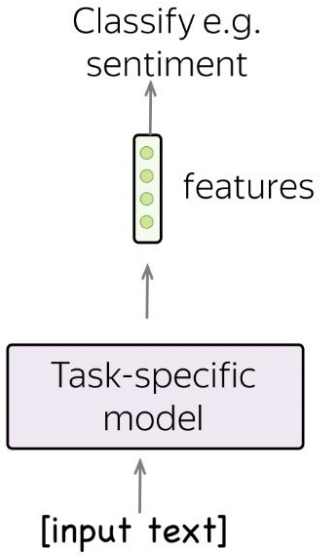
1. LLMs as instruct models
2. Finetuning for instructions
3. Alignment with user feedback
    a. RLHF
    b. Contrastive Learning

Evolutionary journey in NLP

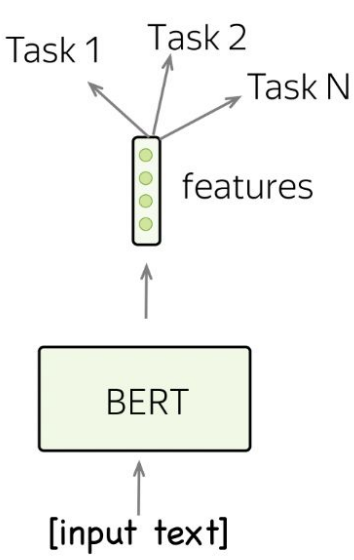
Different task – another set of features (and people!)



Different task – another model



One model, classify anything



Talk to the model

Input (prompt)

What is the sentiment of the next sentence? I love this movie!

Model output

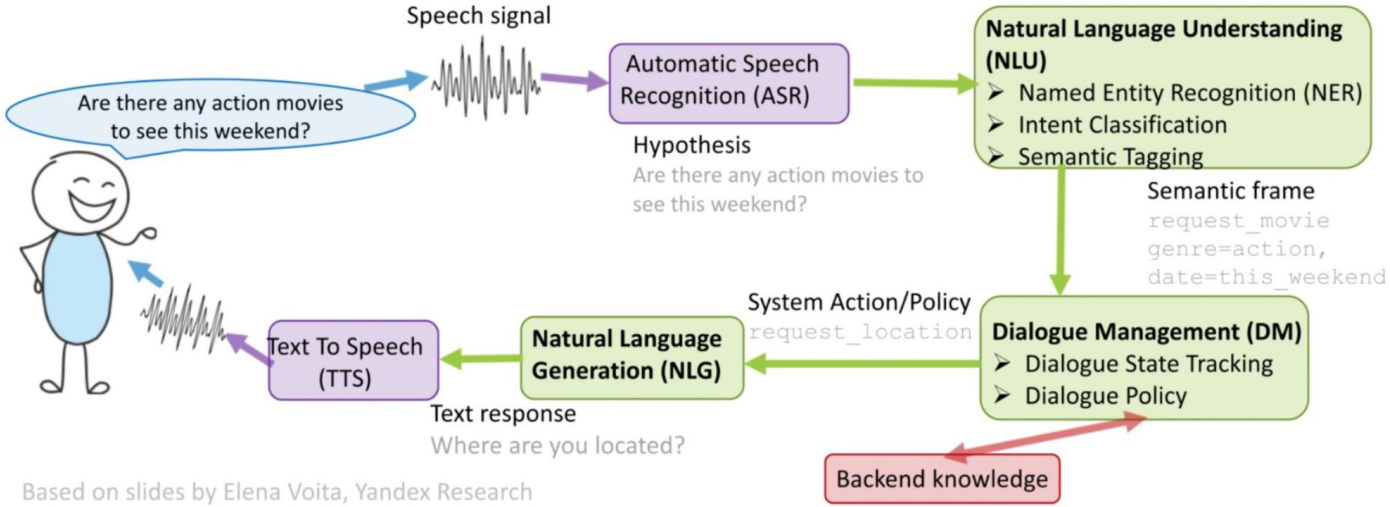
positive

LLMs as dialogue assistants

Dialogue assistants: Cascade pipeline

System design: let's split "chat bot" into smaller problems

Speech processing: see YSDA speech course
Business knowledge: rules for banking / psychology / ...
Text processing: this lecture



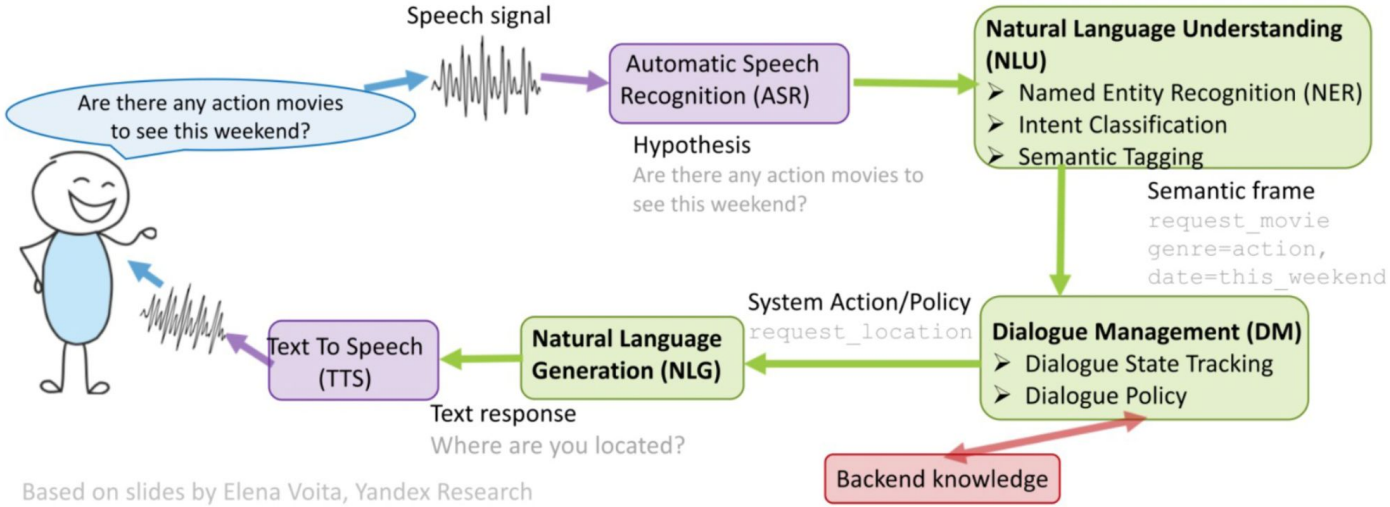
Based on slides by Elena Voita, Yandex Research

Dialogue assistants: Cascade pipeline

Any problems here?

System design: let's split "chat bot" into smaller problems

Speech processing: see YSDA speech course
Business knowledge: rules for banking / psychology / ...
Text processing: this lecture

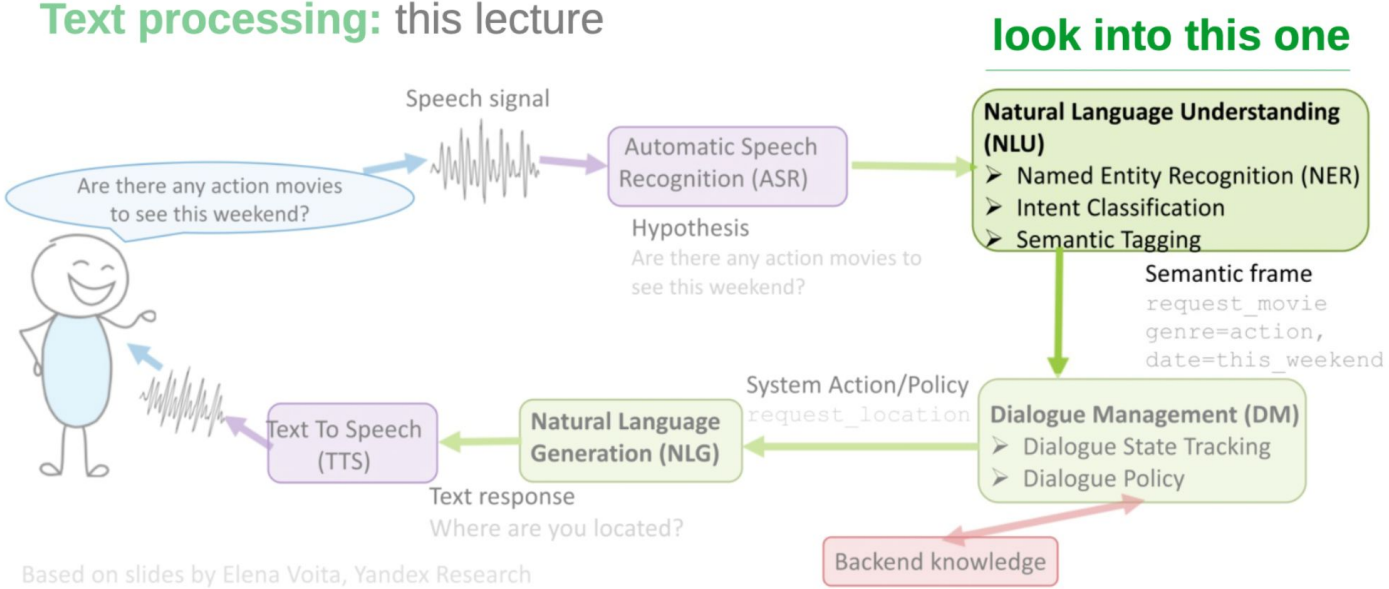


Based on slides by Elena Voita, Yandex Research

Dialogue assistants: Cascade pipeline

System design: let's split "chat bot" into smaller problems

Speech processing: see YSDA speech course
Business knowledge: rules for banking / psychology / ...
Text processing: this lecture



Based on slides by Elena Voita, Yandex Research

Dialogue assistants: Cascade pipeline

Named Entity Recognition

Why: extract keywords from user's message, use them , e.g.
- for web search, when checking a fact
- for map look-up ("where can I buy pizza in Taganrog?")
- to play music / video ("play Eminem's latest song")

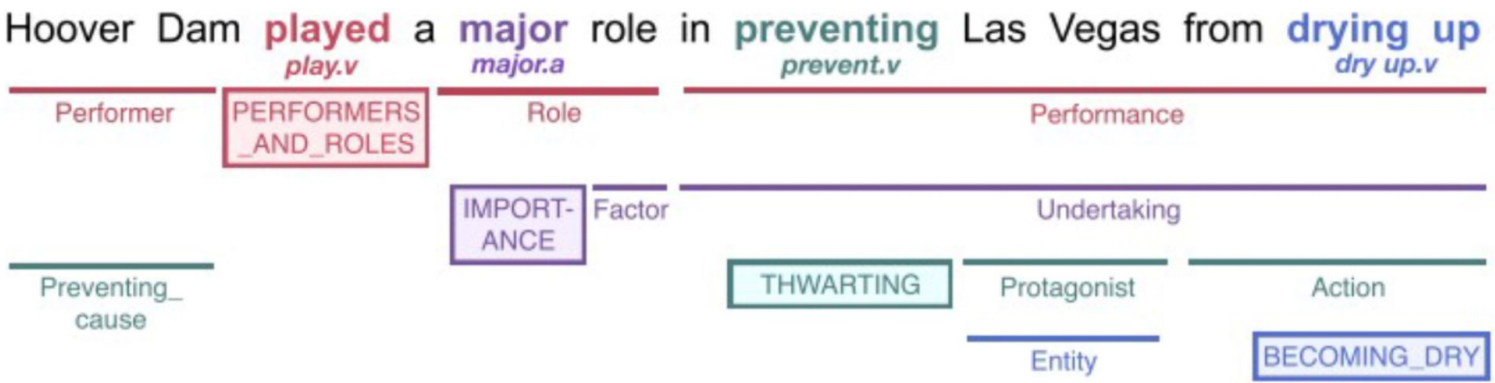
Q: how do we solve this?

When Sebastian Thrun PERSON started working on self - driving cars at Google ORG in 2007 DATE , few people outside of the company took him seriously . " I can tell you very senior CEOs of major American NORP car companies would shake my hand and turn away because I was n't worth talking to , " said Thrun PERSON , in an interview with Recode ORG earlier this week DATED .

Image credit: nanonets.com

Dialogue assistants: Cascade pipeline

- **Named Entity Recognition:** see previous slide
- **Semantic parsing:** to figure out what user wants from you

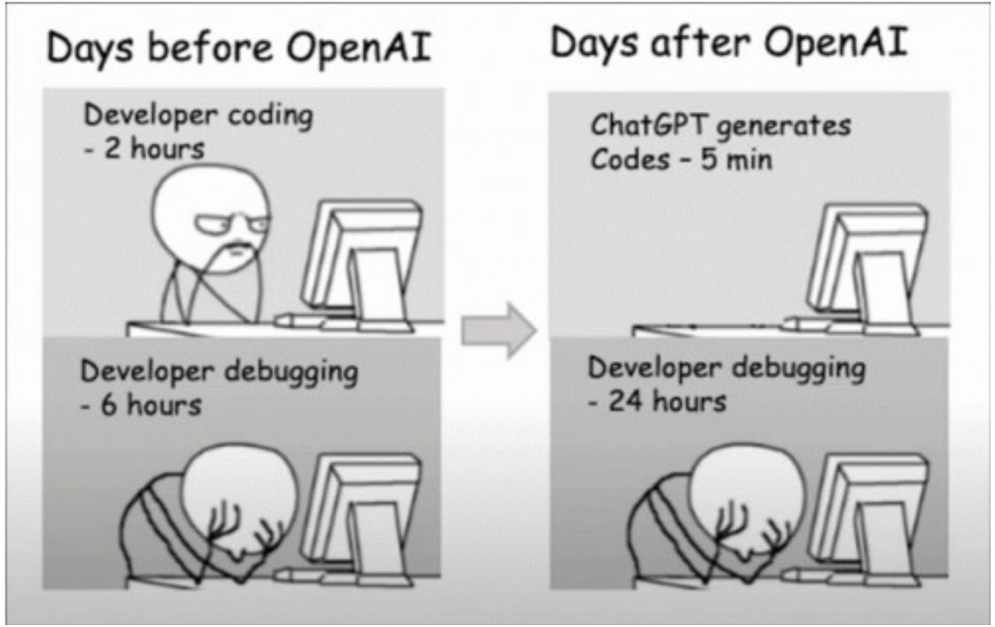


Dialogue assistants: Cascade pipeline

- **Named Entity Recognition:** see previous slide
- **Semantic parsing:** to figure out what user wants from you
- **Anaphora resolution:** to find what “it” or “this bar” refers to

Dialogue assistants: e2e system

the ChatGPT part



GPT-based assistants

GPT pretraining is not aimed at following instructions

Prompt:
What is the purpose of the list C in the code below?

```
def binomial_coefficient(n, r):
     C = [0 for i in range(r + 1)];
     C[0] = 1;
     for i in range(1, n + 1):
          j = min(i, r);
          while j > 0:
               C[j] += C[j - 1];
               j -= 1;
     return C[r]
```

GPT-3 175B completion:
A. to store the value of C[0]
B. to store the value of C[1]
C. to store the value of C[i]
D. to store the value of C[i - 1]

GPT-based assistants

GPT pretraining is not aimed at following instructions

Prompt:
What is the purpose of the list C in the code below?

```
def binomial_coefficient(n, r):
    C = [0 for i in range(r + 1)];
    C[0] = 1;
    for i in range(1, n + 1):
        j = min(i, r);
        while j > 0:
            C[j] += C[j - 1];
            j -= 1;
    return C[r]
```

GPT-3 175B completion:
A. to store the value of C[0]
B. to store the value of C[1]
C. to store the value of C[i]
D. to store the value of C[i - 1]

Solution?

Instruction Tuning

Training objective:

what we **want** vs
what we **told** model to do

What we **told** model to do:
- predict the next token on a webpage from the internet

Alignment →

What we want model to do:
- follow the user's instructions helpfully and safely

The language modeling objective is misaligned

Source: Training language models to follow instructions with human feedback, NeurIPS 2022

Instruction Tuning: InstructGPT

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.

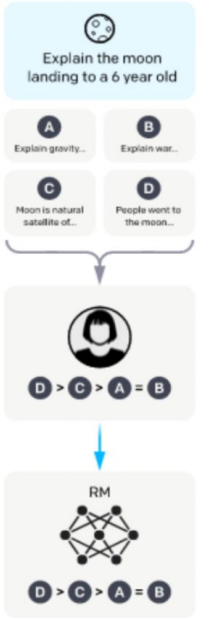
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

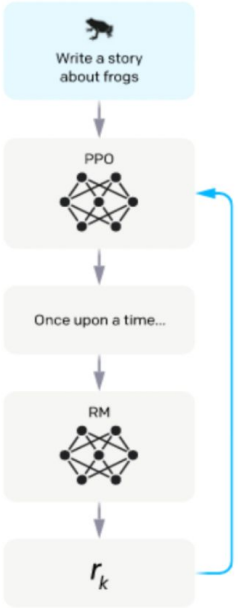
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

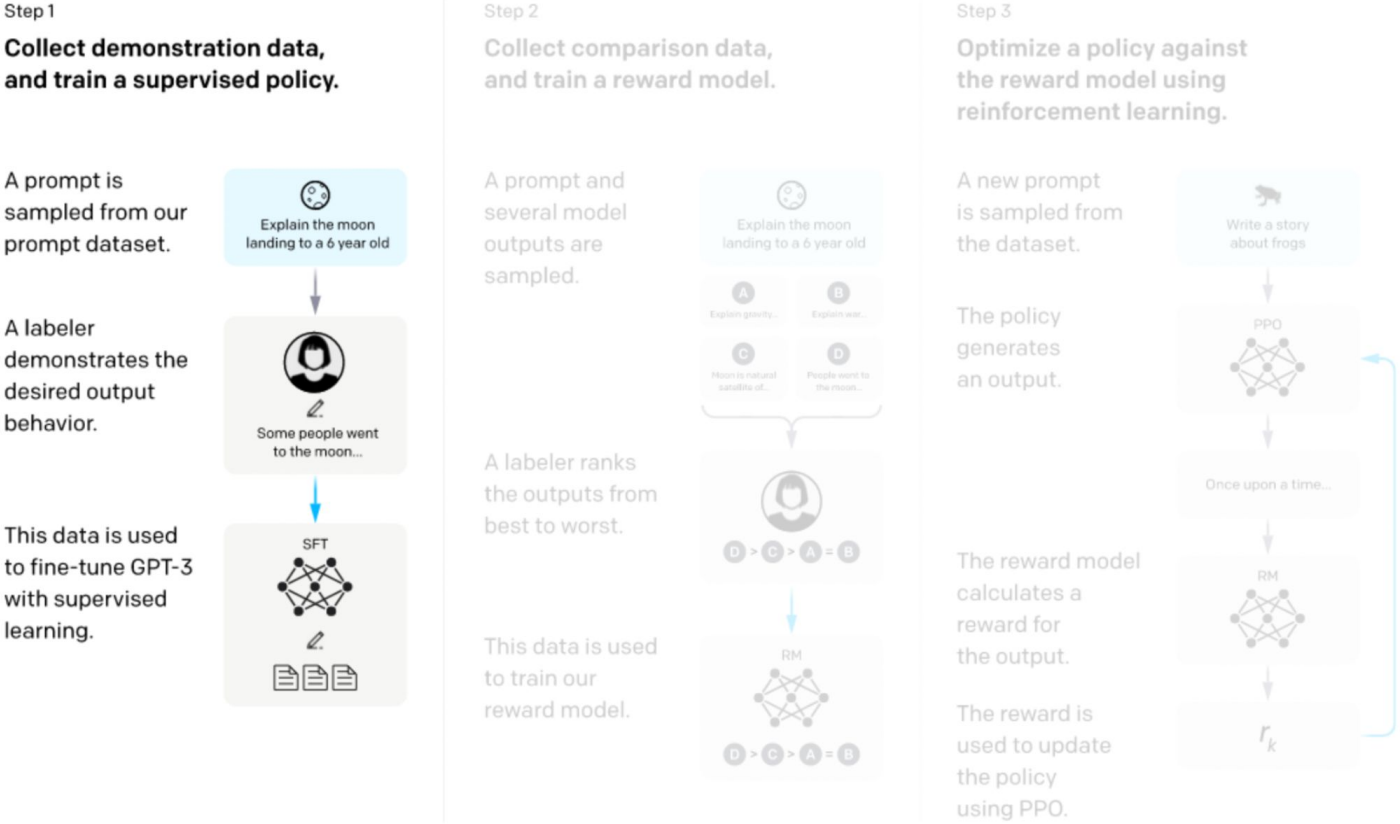
The policy generates an output.

The reward model calculates a reward for the output.

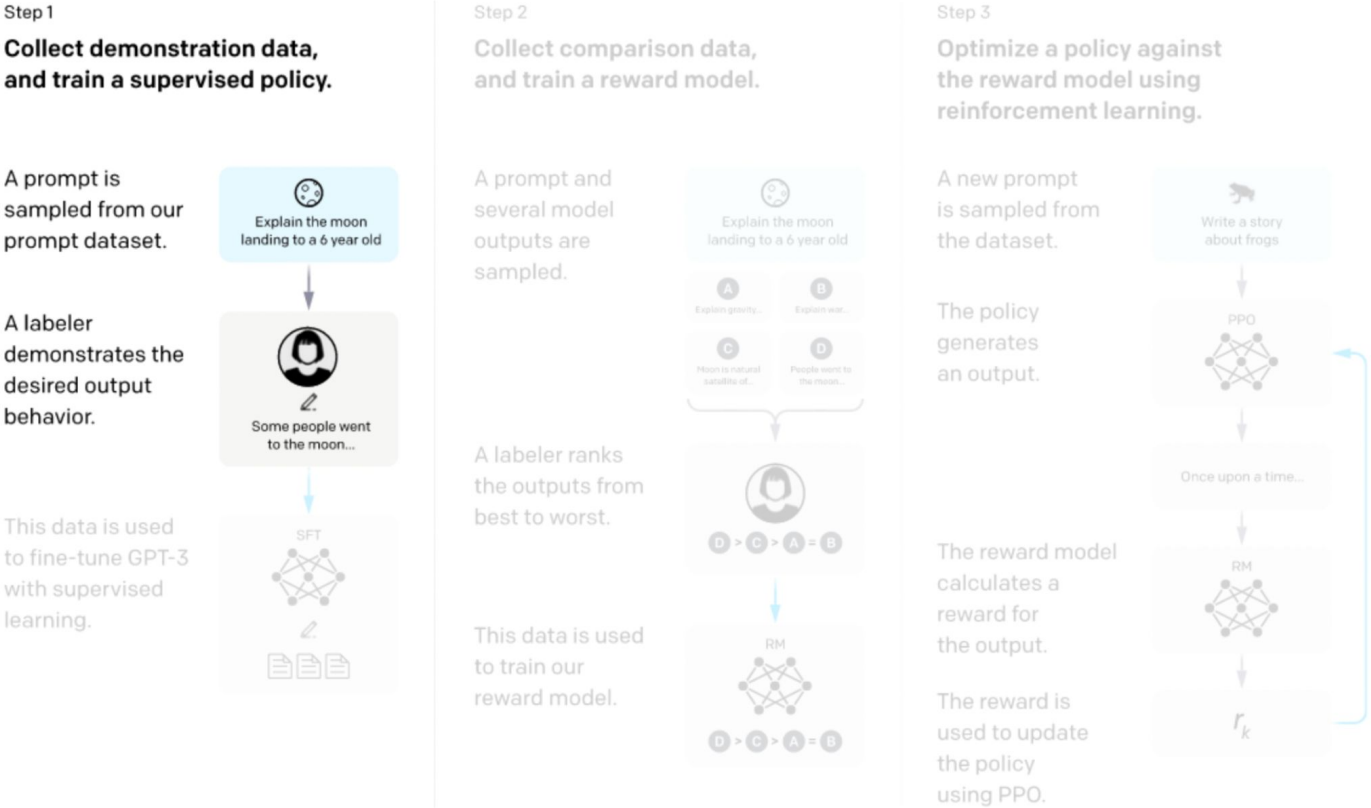
The reward is used to update the policy using PPO.



Instruction Tuning: InstructGPT



Instruction Tuning: InstructGPT



Stage 1: Supervised Finetuning

Prompt Dataset

Initial stage

Manually written by labelers

- Plain: come up with an arbitrary task, while ensuring the tasks had sufficient diversity
- Few-shot: come up with an instruction, and multiple query/response pairs for that instruction
- User-based: come up with prompts corresponding to some of the use-cases stated in waitlist applications to the OpenAI API

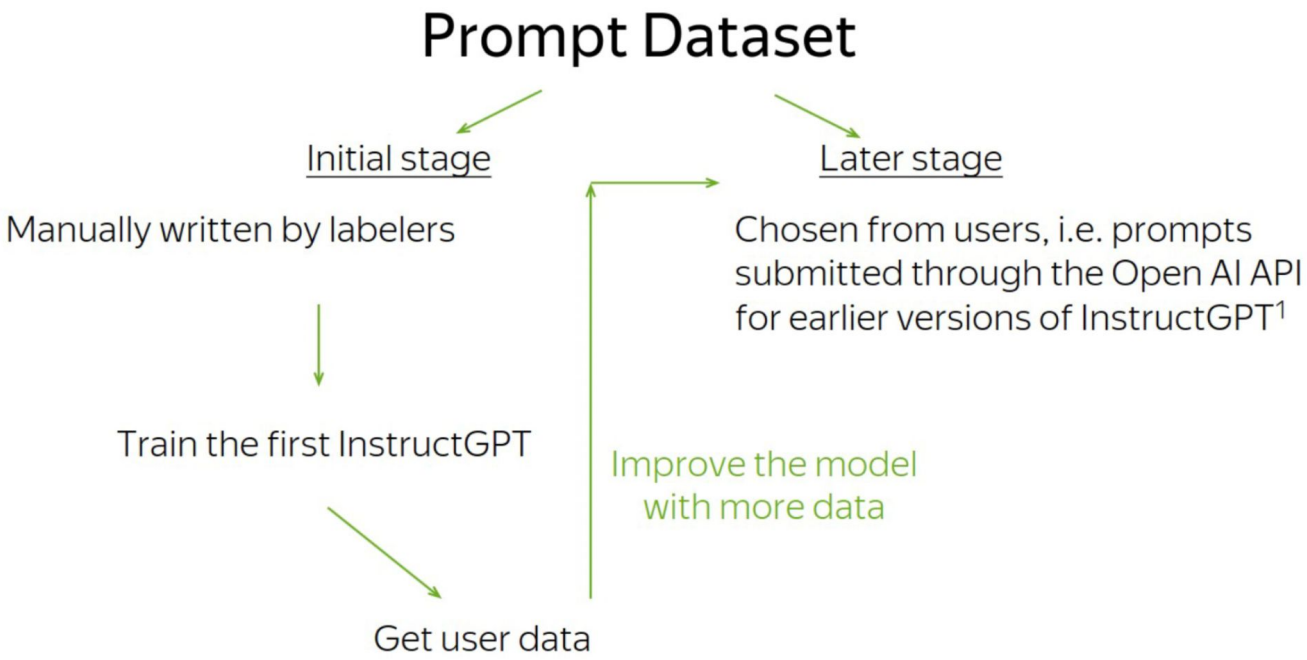
Later stage

Chosen from users, i.e. prompts submitted through the Open AI API for earlier versions of InstructGPT[1]

- filter all prompts in the training split for personally identifiable information (PII)
- heuristically deduplicate prompts
- no more than 200 prompts per user ID
- create train, validation, and test splits based on user ID

[1]https://beta. openai.com/playground

Stage 1: Supervised Finetuning



[1]https://beta. openai.com/playground

Stage 1: Supervised Finetuning

Use cases

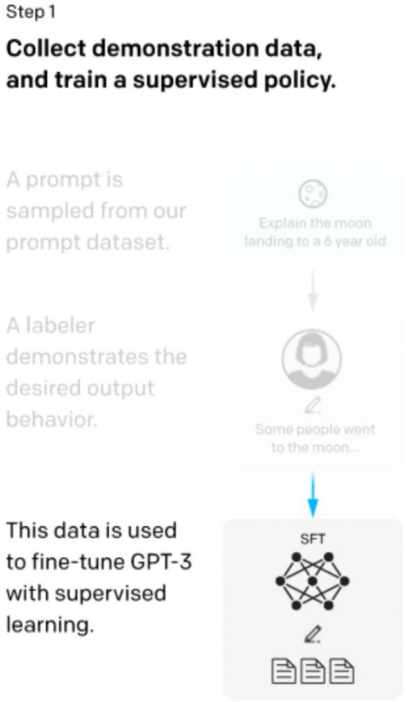
Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

Examples

Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.
Rewrite	This is the summary of a Broadway play: """ {summary} """ This is the outline of the commercial for that play: """

Source: Training language models to follow instructions with human feedback, NeurIPS 2022

Stage 1: Supervised Finetuning



Fine-tuning procedure:

exactly the same as in pre-training

minimize crossentropy with Adam

using the instruction-following data

Cheaper version: train LoRA adapters

https://arxiv.org/abs/2305.14314

InstructGPT

Why can’t we stop at SFT stage?

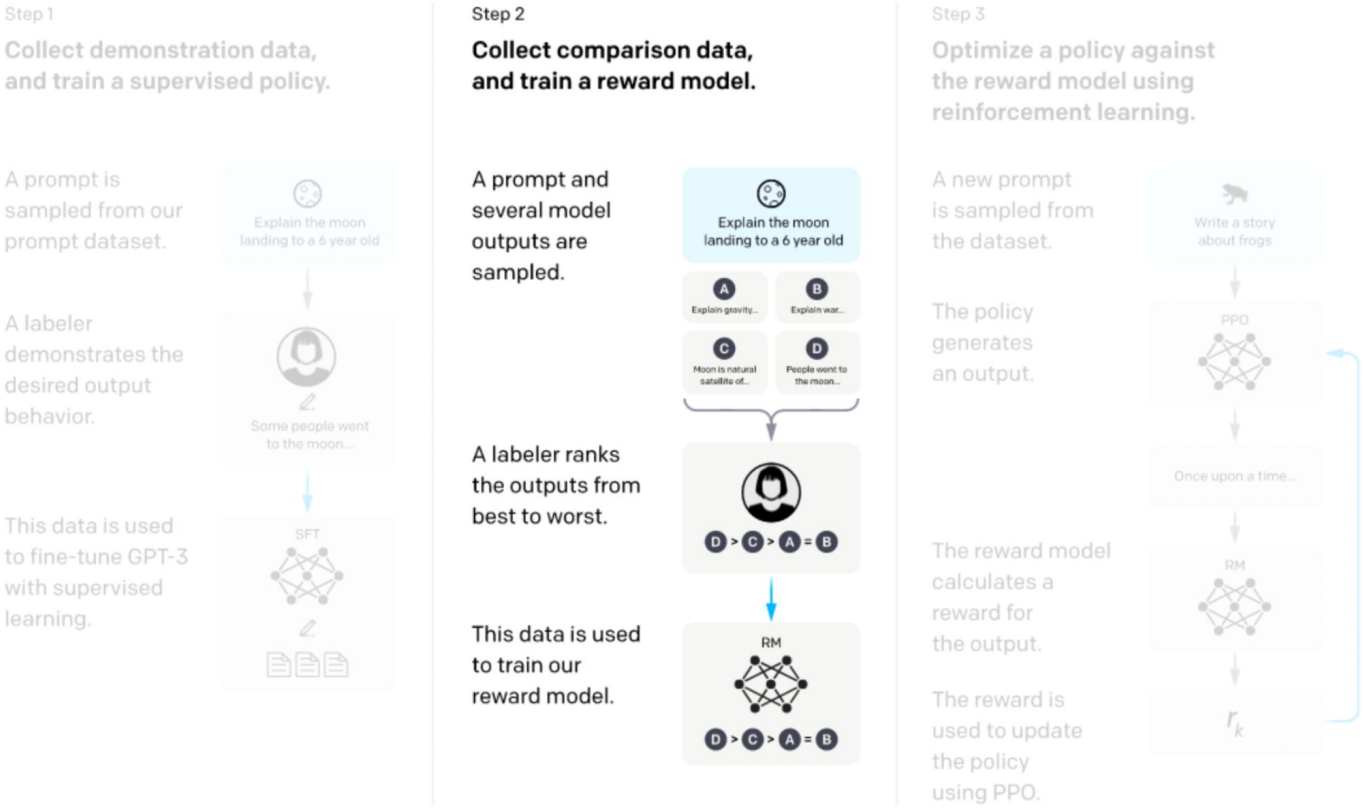
Reason 1: ranking is easier than writing for labelers
(except maybe for fact-checking)

Reason 2: supervised fine-tuning promotes hallucinations

More supervised finetuning problems

- Expensive data collection (hard to achieve answer space coverage)
- Problems with modeling
  a. Xent penalizes all errors equally
  b. Ground truth is technically one-hot -> does not estimate “answer quality” directly
- Exposure bias

Stage 2: Reward Model



Stage 2: Reward Model

How OpenAI did it

Choose:

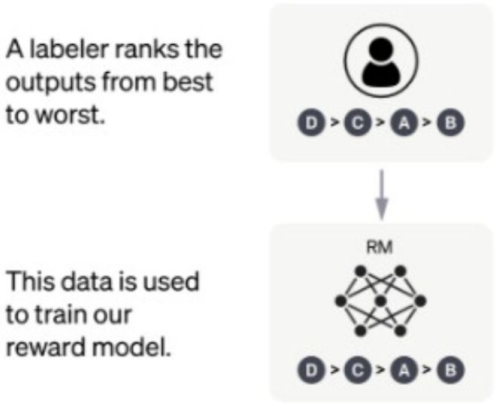
- 40 contractors on Upwork and through ScaleAI
- labelers who were sensitive to the preferences of different demographic groups
- labelers who were good at identifying outputs that were potentially harmful

Mentor:

- Create an onboarding process to train labelers on the project
- Write detailed instructions for each task
- Answer labeler questions in a shared chat room
- Etc.

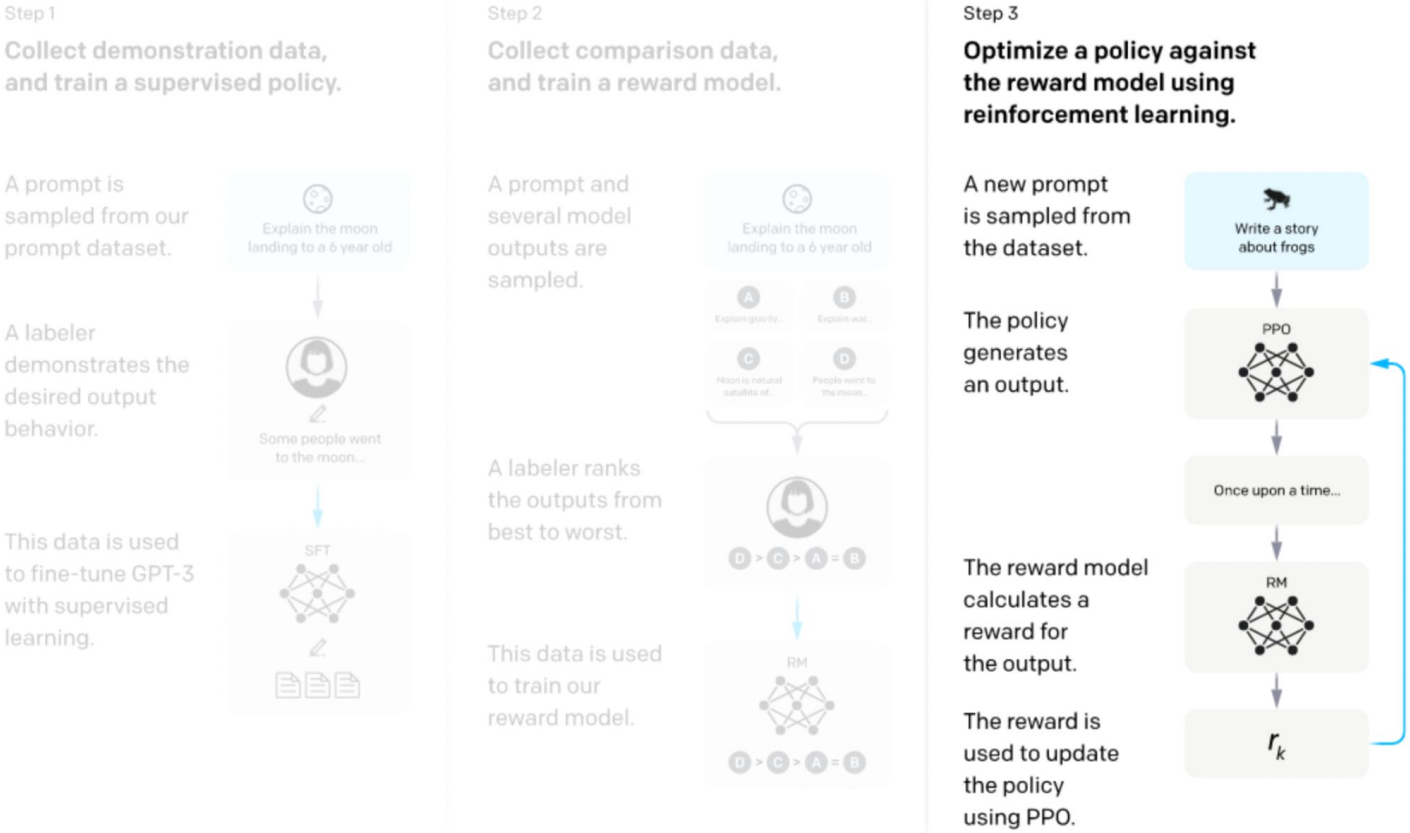
Stage 2: Reward Model

- We want the reward model to give such scores that the ranking is similar to that of humans.
- For every pair the ranking is wrong, the reward model is penalized.



$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x,y_w,y_l)\sim D}\left[\log\left(\sigma\left(r_\theta\left(x,y_w\right) - r_\theta\left(x,y_l\right)\right)\right)\right]$$

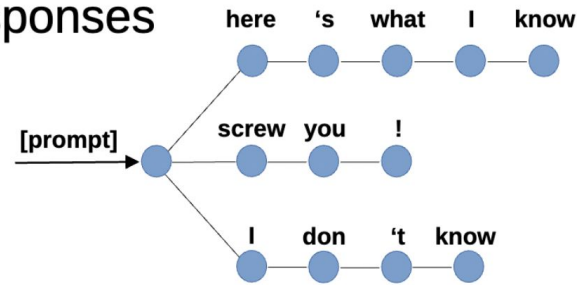
Stage 3: RL Finetuning



Self-critical sequence training

TL;DR reinforcement learning for LLM tasks:

1. Let the model generate several responses (sample with probability)

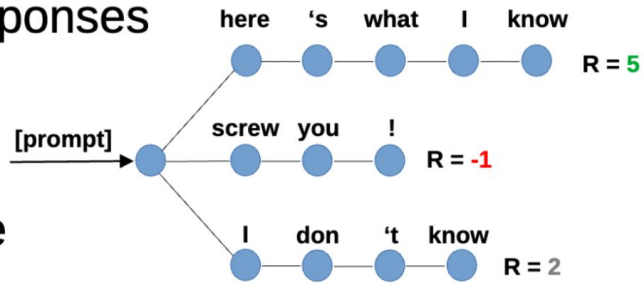


Self-critical sequence training

TL;DR reinforcement learning for LLM tasks:

1. Let the model generate several responses (sample with probability)

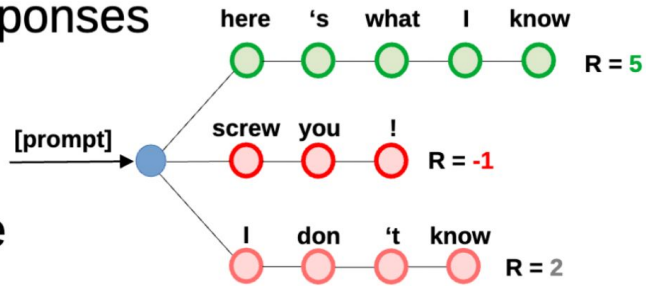
2. Compute reward for each response (apply reward model)



Self-critical sequence training

TL;DR reinforcement learning for LLM tasks:

1. Let the model generate several responses (sample with probability)



2. Compute reward for each response (apply reward model)

3. Train to increase probability of responses **with high reward** (and decrease probability if reward is low)

PPO

1. TRPO (Trust Region Policy Optimization)

Surrogate loss: $\max_{\pi} L(\pi) = \mathbb{E}_{\pi_{\text{old}}} \left[\frac{\pi(a|s)}{\pi_{\text{old}}(a|s)} A^{\pi_{\text{old}}}(s,a) \right]$

Constraint: $\mathbb{E}_{\pi_{\text{old}}} \left[KL(\pi || \pi_{\text{old}}) \right] \leq \epsilon$

for iteration=1, 2, . . . **do**
  Run policy for T timesteps or N trajectories
  Estimate advantage function at all timesteps
  Compute policy gradient g
  Use CG (with Hessian-vector products) to compute $F^{-1}g$
  Do line search on surrogate loss and KL constraint
end for

PPO

TRPO

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}\hat{A}_t\right]$$

$$\text{subject to} \quad \hat{\mathbb{E}}_t[\mathrm{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_\theta(\cdot \mid s_t)]] \leq \delta$$

PPO v1

$$\max_\theta \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}\hat{A}_t\right] - \beta\left(\hat{\mathbb{E}}_t\left[\mathrm{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid s_t), \pi_\theta(\cdot \mid s_t)]\right] - \delta\right)$$

- Pseudocode:

  for iteration=1, 2, . . . **do**

  Run policy for T timesteps or N trajectories

  Estimate advantage function at all timesteps

  Do SGD on above objective for some number of epochs

  Do dual descent update for beta

InstructGPT

Prompt:
What is the purpose of the list C in the code below?

```
def binomial_coefficient(n, r):
     C = [0 for i in range(r + 1)];
     C[0] = 1;
     for i in range(1, n + 1):
          j = min(i, r);
          while j > 0:
               C[j] += C[j - 1];
               j -= 1;
     return C[r]
```

GPT-3 175B completion:
A. to store the value of C[0]
B. to store the value of C[1]
C. to store the value of C[i]
D. to store the value of C[i - 1]

InstructGPT 175B completion:
The list C in this code is used to store the values of the binomial coefficient as the function iterates through the values of n and r. It is used to calculate the value of the binomial coefficient for a given value of n and r, and stores the result in the final return value of the function.

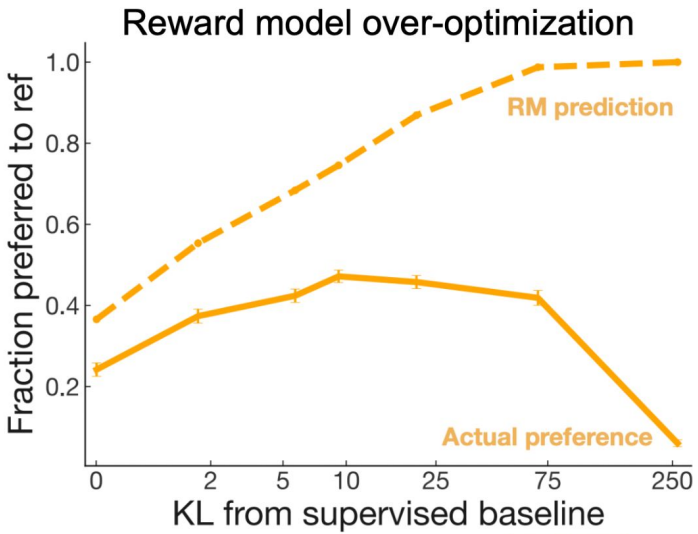
RLHF: Reward Modeling (RM)

Why do we need RM?

1. RL is sample expensive
    a. either A LOT of human evaluations needed
    b. or - train Reward Model on small corpus, train on big synthetic corpus
2. PPO if on-policy
    a. training samples for PPO need to come from distribution of current stage of model
    b. human evaluations are pre-collected -> off-policy
    c. reward model scores can be computed on policy

Problems of RLHF

- Human preferences are unreliable!
  - ”Reward hacking” is a common problem in RL
  - Chatbots are rewarded to produce responses that *seem* authoritative and helpful, *regardless of truth*
  - This can result in making up facts + hallucinations
- **Models** of human preferences are *even more* unreliable!

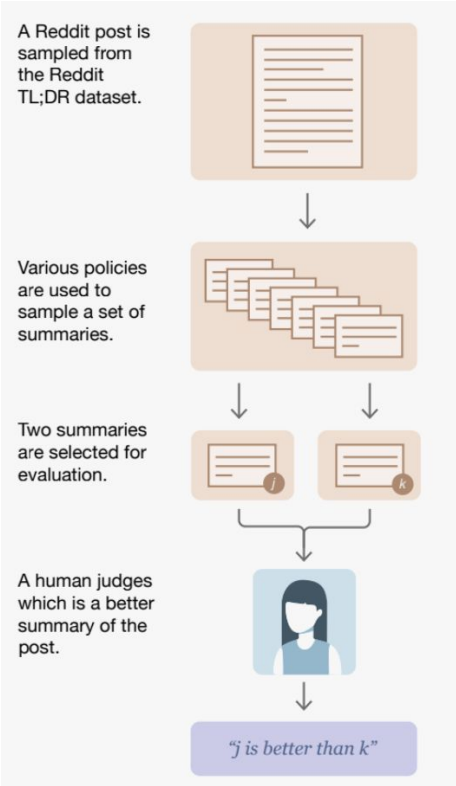


$$R(s) = RM_\phi(s) - \beta \log \left(\frac{p_\theta^{RL}(s)}{p^{PT}(s)} \right)$$

[Stiennon et al., 2020]

Learning from Human Feedback: any alternatives?

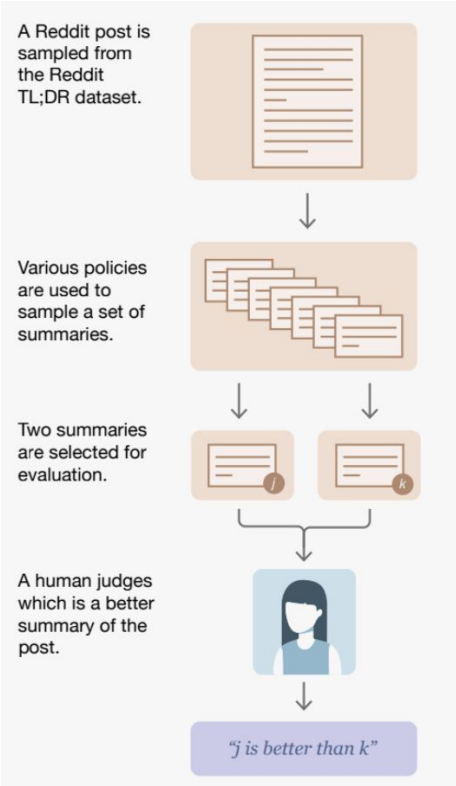
<u>Reminder:</u> main goal is to utilize answers ranking from humans



Learning from Human Feedback: any alternatives?

<u>Reminder:</u> main goal is to utilize answers ranking from humans

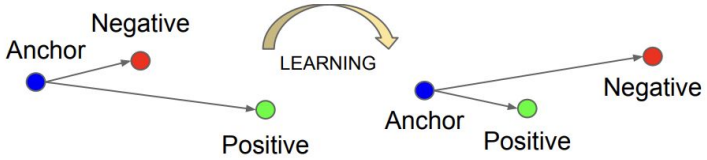
Contrastive Learning!



DSSM

DSSM (Deep Structured Semantic Model)

Textual data	→	Text encoder (word2vec, RNN etc.)	*A (anchor)*
Corresponding image	→	Image encoder (CNN etc.)	*P (positive)*
Non-correspon ding image	→	Image encoder (CNN etc.)	*N (negative)*



Triplet loss (max margin loss)

Goal: $\|f(x_i^a) - f(x_i^p)\|_2^2 + \alpha < \|f(x_i^a) - f(x_i^n)\|_2^2$

Loss: $\sum_i^N \left[\|f(x_i^a) - f(x_i^p)\|_2^2 - \|f(x_i^a) - f(x_i^n)\|_2^2 + \alpha\right]_+$

Triplet loss for seq2seq tasks

$$\mathcal{L}(\theta) = max(0, \delta - \log P_\theta(\mathbf{y}^+|\mathbf{x}) + \log P_\theta(\mathbf{y}^-|\mathbf{x}))$$

Triplet loss for seq2seq tasks

$$\mathcal{L}(\theta) = max(0, \delta - \log P_\theta(\mathbf{y}^+|\mathbf{x}) + \log P_\theta(\mathbf{y}^-|\mathbf{x}))$$

response with higher rating

prompt / dialogue state

response with lower rating

SLiC-HF

Contrastive Learning from Human Feedback

1. SFT model
2. Collect off-policy dataset with HF
3. **S**equence **Li**kelihood **C**alibration with **H**uman **F**eedback

SLiC-HF

Contrastive Learning from Human Feedback

1. SFT model
2. Collect off-policy dataset with HF
3. **S**equence **Li**kelihood **C**alibration with **H**uman **F**eedback

$$\mathcal{L}(\theta) = \underbrace{\sum L^{\text{cal}}(\theta, \mathbf{x}, \mathbf{y}_{\text{ref}}, \{\hat{\mathbf{y}}\}_m)}_{\textit{contrastive triplet loss}} + \underbrace{\lambda L^{\text{reg}}(\theta, \theta_{ft}; \mathbf{x}, \mathbf{y}_{\text{ref}})}_{\textit{Xent regularizer}}$$

SLiC-HF

Contrastive Learning from Human Feedback

1. SFT model
2. Collect off-policy dataset with HF
3. **S**equence **Li**kelihood **C**alibration with **H**uman **F**eedback

$$\mathcal{L}(\theta) = \underbrace{\sum L^{\text{cal}}(\theta, \mathbf{x}, \mathbf{y}_{\text{ref}}, \{\hat{\mathbf{y}}\}_m)}_{\textit{contrastive triplet loss}} + \underbrace{\lambda L^{\text{reg}}(\theta, \theta_{ft}; \mathbf{x}, \mathbf{y}_{\text{ref}})}_{\textit{Xent regularizer}}$$

$$\mathcal{L}(\theta) = max(0, \delta - \log P_\theta(\mathbf{y}^+|\mathbf{x}) + \log P_\theta(\mathbf{y}^-|\mathbf{x})) - \lambda \log P_\theta(\mathbf{y}_{\text{ref}}|\mathbf{x})$$

DPO

DPO (Direct Preference Optimization)

1. SFT model
2. Collect off-policy dataset with HF
3. DPO contrastive finetuning

DPO

DPO (Direct Preference Optimization)

1. SFT model
2. Collect off-policy dataset with HF
3. DPO contrastive finetuning

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)\sim\mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]$$

- Triplet max-margin loss -> logsigmoid
- Raw logP diff -> relative logP w.r.t. ref model diff

GRPO

GRPO (Group Relative Preference Optimization)

1. SFT model
2. On-policy training
3. GRPO loss – variation of PPO

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{\sum_{i=1}^{G} |o_i|} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} \left[\frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{[\pi_\theta(o_{i,t} \mid q, o_{i,<t})]_{\text{no grad}}} \hat{A}_{i,t} - \beta \mathbb{D}_{\text{KL}} \left[\pi_\theta \| \pi_{\text{ref}} \right] \right]$$

Contrastive Learning vs. RLHF

CL advantages:

- More data efficient; RL requires a lot of episodes for convergence
- More stable: PPO updates are noisy and can lead to reward hacking

CL disadvantages:

- Heavily reliant on data; can easily overfit
- If base model is of very poor quality, unstable convergence

Questions?